Report

Part 1: Service Architecture & Setup

Question 1.1: Explain the default lifecycle of a JAX-RS Resource class. Is a new instance instantiated for every incoming request, or does the runtime treat it as a singleton? Elaborate on how this architectural decision impacts the way you manage and synchronize your in-memory data structures (maps/lists) to prevent data loss or race conditions.

By default, JAX-RS resource classes follow a **per-request lifecycle**. This means a new instance of the resource class is instantiated by the JAX-RS runtime for every incoming HTTP request, and that instance is discarded after the response is sent. The resource class is **not** treated as a singleton by default.

This architectural decision has significant implications for how in-memory data structures are managed. Since each request gets a fresh resource instance, any instance-level fields (e.g., a `HashMap` or `ArrayList` declared as instance variables) would be created anew with each request, losing all previously stored data. To persist data across requests, the data store must exist **outside** the per-request lifecycle — typically as a **singleton**. In this project, the `DataStore` class uses the Singleton pattern (`private static final DataStore INSTANCE`) to maintain a single shared instance across all requests.

Furthermore, because multiple requests can be processed concurrently by different threads (each with their own resource instance but sharing the same `DataStore`), thread safety is critical. This is why `ConcurrentHashMap` is used instead of a regular `HashMap` — it provides thread-safe operations without requiring explicit synchronization, preventing race conditions such as lost updates or corrupted reads when multiple clients simultaneously modify the data.

Question 1.2: Why is the provision of "Hypermedia" (links and navigation within responses) considered a hallmark of advanced RESTful design (HATEOAS)? How does this approach benefit client developers compared to static documentation?

The provision of "Hypermedia" — embedding navigational links within API responses — is considered a hallmark of advanced RESTful design because it enables **discoverability** and **self-documentation** of the API. This principle, known as HATEOAS (Hypermedia as the Engine of Application State), means clients can navigate the entire API by following links provided in responses rather than hard-coding endpoint URLs.

Benefits for client developers compared to static documentation include:

1. **Reduced coupling**: Clients do not need to construct URLs manually; they follow links from responses, making the API more resilient to URL structure changes.
2. **Dynamic navigation**: The server can conditionally include or exclude links based on the current state of a resource (e.g., only showing a "delete" link when deletion is permitted), guiding clients through valid transitions.
3. **Simplified onboarding**: New developers can explore the API starting from the root discovery endpoint, following links to discover available resources without needing to read extensive external documentation.

4. **Version resilience**: If the API evolves and paths change, clients following links automatically adapt, reducing maintenance burden.

The discovery endpoint at `GET /api/v1` in this project exemplifies this by providing a map of primary resource collections, allowing clients to discover `rooms` and `sensors` endpoints dynamically.

Part 2: Room Management

Specification Compliance Note: Part 2 requires a "RoomResource" class (not "SensorRoom") to manage the `/api/v1/rooms` path with GET, POST, and DELETE operations plus business logic constraints.

Question 2.1: When returning a list of rooms, what are the implications of returning only IDs versus returning the full room objects? Consider network bandwidth and client-side processing.

When returning a list of rooms, there are trade-offs between returning only IDs and returning full room objects:

Returning only IDs:

- **Advantage**: Significantly reduces payload size and network bandwidth consumption, especially when the collection is large (thousands of rooms). The response is lightweight and fast to transmit.
- **Disadvantage**: Forces the client to make additional HTTP requests (one per room) to fetch the full details of each room. This creates an "N+1 query" problem on the client side, increasing overall latency and server load due to the high number of round-trips.

Returning full room objects (current implementation):

- **Advantage**: The client receives all the information it needs in a single request. This reduces the total number of HTTP calls and simplifies client-side processing, as no additional requests are required to display room details.
- **Disadvantage**: The response payload is larger, consuming more bandwidth. For very large collections, this could lead to slow response times and high memory usage.

For this campus management API, returning full objects is the pragmatic choice because the room data model is relatively small and facilities managers typically need to see all room details at once. For much larger datasets, pagination or a hybrid approach (returning summary fields with links to full details) would be more appropriate.

Question 2.2: Is the DELETE operation idempotent in your implementation? Provide a detailed justification by describing what happens if a client mistakenly sends the exact same DELETE request for a room multiple times.

Yes, the `DELETE` operation in this implementation is **idempotent**. Idempotency means that making the same request multiple times produces the same result (server state) as making it once.

Here is what happens if a client mistakenly sends the exact same `DELETE` request for a room multiple times:

1. **First request**: The room exists and has no sensors. The room is removed from the data store. The server returns `204 No Content`, indicating successful deletion.
2. **Second (and subsequent) identical requests**: The room no longer exists in the data store. The server detects this and returns a `404 Not Found` response with an error message stating the room was not found.

The key point is that the **server-side state remains the same** after the first successful deletion — the room is gone and stays gone. Although the HTTP response code changes from `204` to `404` on subsequent calls, the server state does not change, which satisfies the definition of idempotency. The `404` response is not an error in terms of state mutation; it simply informs the client that the resource does not exist.

Part 3: Sensor Operations & Linking

Question 3.1: We explicitly use the @Consumes(MediaType.APPLICATION_JSON) annotation on the POST method. Explain the technical consequences if a client attempts to send data in a different format, such as text/plain or application/xml. How does JAX-RS handle this mismatch?

The `@Consumes(MediaType.APPLICATION_JSON)` annotation explicitly declares that the POST method only accepts request bodies formatted as JSON. When a client attempts to send data in a different format, the following technical consequences occur:

- **JAX-RS Content Negotiation**: The JAX-RS runtime inspects the `Content-Type` header of the incoming request. If the client sends `text/plain` or `application/xml`, the runtime cannot find a matching resource method that consumes that media type.
- **HTTP 415 Unsupported Media Type**: JAX-RS automatically returns an HTTP `415 Unsupported Media Type` response without ever invoking the resource method. This is handled entirely by the framework before the method body executes.
- **No custom handling required**: The developer does not need to write any validation code to check the content type — JAX-RS enforces this constraint declaratively through the annotation.

This is a key advantage of declarative annotations in JAX-RS: the framework handles protocol-level validation (content type negotiation) transparently, allowing developers to focus on business logic while ensuring the API strictly adheres to its contract.

Question 3.2: You implemented this filtering using @QueryParam. Contrast this with an alternative design where the type is part of the URL path (e.g., /api/v1/sensors/type/CO2). Why is the query parameter approach generally considered superior for filtering and searching collections?

The `@QueryParam` approach (`/api/v1/sensors?type=CO2`) is generally considered superior to the path-based approach (`/api/v1/sensors/type/CO2`) for filtering and searching collections for several reasons:

1. **Semantic clarity**: In REST, URL paths identify **resources**. `/api/v1/sensors/type/CO2` implies that `type` and `CO2` are nested resources, which is semantically incorrect — `CO2` is not a sub-resource of sensors; it is a filter criterion. Query parameters clearly express that the client is requesting a filtered view of the **same** resource collection.

2. **Combinability**: Query parameters naturally support multiple filters (e.g., `?type=CO2&status=ACTIVE`). Path-based filtering becomes increasingly awkward and unscalable with multiple criteria (`/sensors/type/CO2/status/ACTIVE`), creating a combinatorial explosion of path patterns.

3. **Optionality**: Query parameters are inherently optional. The same endpoint (`GET /sensors`) works with or without the `?type=` parameter. With path-based filtering, you would need separate route definitions for filtered and unfiltered access.

4. **HTTP caching**: Properly structured query strings work well with HTTP caching mechanisms and proxies, which understand query parameters as variations of the same resource.

5. **Convention**: The HTTP specification and REST conventions establish that query strings are the standard mechanism for parameterizing resource retrieval (searching, filtering, sorting, pagination).

Part 4: Deep Nesting with Sub-Resources

Question 4.1: Discuss the architectural benefits of the Sub-Resource Locator pattern. How does delegating logic to separate classes help manage complexity in large APIs compared to defining every nested path (e.g., sensors/{id}/readings/{rid}) in one massive controller class?

The Sub-Resource Locator pattern provides several architectural benefits for managing complexity in large APIs:

1. **Separation of Concerns**: Each resource class handles only the logic relevant to its specific resource type. `SensorResource` handles sensor CRUD operations, while `SensorReadingResource` handles reading-specific logic. This is far cleaner than a single monolithic controller class containing dozens of methods for every nested path.

2. **Encapsulation**: The sub-resource class encapsulates all knowledge about how to manage readings for a specific sensor. The parent `SensorResource` only needs to validate the sensor exists and delegate — it does not need to know the details of reading management.

3. **Reusability**: Sub-resource classes can potentially be reused in different contexts or mounted under different parent resources without code duplication.

4. **Maintainability**: When the readings API needs to be extended (e.g., adding aggregation endpoints, pagination, or date-range filtering), changes are isolated to the `SensorReadingResource` class. Developers can work on different resource classes independently without merge conflicts or risk of breaking unrelated functionality.

5. **Testability**: Smaller, focused classes are significantly easier to unit test. Each sub-resource class can be tested in isolation with mock data, rather than requiring complex test setups for a massive controller.

6. **Scalability**: As the API grows (e.g., adding alerts, maintenance logs, or calibration history as sub-resources of sensors), each new feature gets its own class. A single massive controller class would become unmaintainable as the number of endpoints grows.

In comparison, defining every nested path (`sensors/{id}/readings`, `sensors/{id}/readings/{rid}`, `sensors/{id}/alerts`, etc.) in one controller class would violate the Single Responsibility Principle and create a "god class" that is difficult to understand, test, and maintain.

Part 5: Advanced Error Handling, Exception Mapping & Logging

Question 5.1: Why is HTTP 422 often considered more semantically accurate than a standard 404 when the issue is a missing reference inside a valid JSON payload?

HTTP `422 Unprocessable Entity` is often considered more semantically accurate than `404 Not Found` when the issue is a missing reference inside a valid JSON payload for the following reasons:

- **404 means the target resource was not found**: A `404` response indicates that the URL the client requested does not map to any existing resource. In this scenario, the client is sending a `POST` to `/api/v1/sensors`, which **does** exist — the URL is correct and the endpoint is valid.
- **422 means the request body is semantically invalid**: The JSON payload is syntactically well-formed and the server can parse it successfully, but the **content** is semantically wrong — it references a `roomId` that does not exist. The server understands the request but **cannot process** it due to a logical/semantic error in the entity body.
- **The distinction matters**: Using `404` would confuse clients into thinking the sensors endpoint does not exist, when in reality the problem is with the **data** they submitted, not the **URL** they requested. `422` correctly communicates: "I received your request, I understand the format, but the content contains a reference to a resource that does not exist in the system."

This aligns with the HTTP specification where `4xx` errors indicate client-side problems — `422` specifically indicates the server understood the content type and syntax but the semantic content was invalid.

Question 5.2: From a cybersecurity standpoint, explain the risks associated with exposing internal Java stack traces to external API consumers. What specific information could an attacker gather from such a trace?

Exposing internal Java stack traces to external API consumers poses significant cybersecurity risks:

1. **Technology fingerprinting**: Stack traces reveal the programming language (Java), framework (Jersey/JAX-RS), server version, and library dependencies. Attackers can use this information to search for known vulnerabilities (CVEs) specific to those versions.
2. **Internal architecture exposure**: Package names (e.g., `com.smartcampus.resource.SensorResource`) reveal the internal package structure, class names, and method names, providing attackers with a detailed map of the application's architecture.
3. **File path disclosure**: Stack traces may include absolute file paths on the server, revealing the operating system, deployment directory structure, and username information.

4. **Logic disclosure**: Method names and class hierarchies in stack traces can reveal business logic flow, helping attackers understand how the application processes requests and where vulnerabilities might exist.

5. **Database/data layer leaks**: If a database-related exception occurs, the stack trace might expose table names, column names, query structures, or connection strings.

6. **Attack surface mapping**: By intentionally triggering different errors, an attacker can systematically collect stack traces to build a comprehensive map of the application's attack surface, identifying input validation weaknesses or injection points.

The global `ExceptionMapper<Throwable>` in this project acts as a safety net by intercepting all unhandled exceptions and returning a generic `500 Internal Server Error` message with no internal details. The actual exception is logged server-side for debugging, but clients only see a sanitized error response.

Question 5.3: Why is it advantageous to use JAX-RS filters for cross-cutting concerns like logging, rather than manually inserting Logger.info() statements inside every single resource method?

Using JAX-RS filters (`ContainerRequestFilter` and `ContainerResponseFilter`) for cross-cutting concerns like logging offers several advantages over manually inserting `Logger.info()` statements in every resource method:

1. **Separation of Concerns (AOP)**: Logging is a cross-cutting concern that should not be mixed with business logic. Filters cleanly separate logging infrastructure from resource method code, following the Aspect-Oriented Programming principle.

2. **DRY (Don't Repeat Yourself)**: Without filters, every resource method would need identical logging boilerplate code. Filters implement logging logic **once** and it automatically applies to **all** endpoints, including any new endpoints added in the future.

3. **Consistency**: Filters guarantee that every request and response is logged in the same format. Manual logging is error-prone — developers may forget to add it, use inconsistent formats, or log different information in different methods.

4. **Maintainability**: If the logging format needs to change (e.g., adding correlation IDs or switching to structured JSON logging), only the single filter class needs modification rather than updating every resource method.

5. **Completeness**: Filters intercept the request **before** any resource method executes and the response **after** it completes. This captures the full lifecycle, including cases where the request is rejected before reaching a resource method (e.g., 404s, 415s).

6. **Zero impact on resource code**: Resource classes remain focused purely on business logic with no logging clutter, improving readability and maintainability.